Linienverfolgung – Programmierkonzept

Nov 5, 2026 · @Lukas

Ziel und Strecke

Der Roboter soll einer 45 mm breiten schwarzen Linie auf weißem Untergrund selbstständig folgen, auch durch Kurven und über die Kreuzung, ohne die Linie zu verlieren.

Die Teststrecke ist eine geschlossene Schleife. An einer Stelle kreuzen sich zwei Linienstücke als **X**, und genau in der Mitte der Kreuzung ist die Linie unterbrochen (eine Lücke). Daraus ergeben sich drei Situationen, die das Programm unterscheiden muss:

- **Normale Linie und Kurven:** Linie unter dem Sensor halten, sanft gegenlenken.
- **Kreuzung mit Lücke:** Für ein kurzes Stück ist keine Linie zu sehen. Der Roboter darf hier nicht suchen oder abbiegen, sondern muss geradeaus weiterfahren, bis er die Linie auf der anderen Seite wiederfindet.
- **Linie wirklich verloren:** Etwa nach einer zu engen Kurve. Dann gezielt in die Richtung suchen, in der die Linie zuletzt war.

Die Hinderniserkennung (Bremsen ab etwa 5 cm) und die BLE-Steuerung bleiben während der Linienfolge aktiv.

Genutzte Bibliotheken

Die Linienfolge baut fast vollständig auf unseren vorhandenen Bibliotheken auf. Neu kommt nur `LineFollower` dazu, die die Logik bündelt.

Bibliothek	Aufgabe in der Linienfolge	Was wir daraus nutzen
`LineSensor`	Wo liegt die Linie?	`read()` für links/Mitte/rechts, `getPosition()` (−1 … 0 … +1). Pegel: LOW = schwarze Linie
`DriveController`	Lenkbefehle an die Motoren	`drive(links, rechts)` in %, `stop()`, `brake()`, `setSpeedScale()` für die Hindernisdrosselung
`MotorDriver`	PWM an den L298N	indirekt über `DriveController`; Mindest-PWM 35 %, Maximum 80 %

Bibliothek	Aufgabe in der Linienfolge	Was wir daraus nutzen
`WheelSpeedSensor`	Gefahrene Strecke messen	`getPulseCounts()`: 1 Puls ≈ 10 mm. Damit wird die Lücke an der Kreuzung überbrückt und das Suchen begrenzt
`UltrasonicSensor`	Hindernis vor dem Roboter	unverändert über `handleObstacle()` in `main.cpp`: drosseln ab 10 cm, bremsen bei ~5 cm
`BleUart`	Bedienung vom Handy	Linienfolge starten/stoppen, Parameter live einstellen, Zustand und Werte mitlesen
`LineFollower` (neu)	Die eigentliche Linienfolge	Lenkregelung und Zustandsautomat (siehe nächster Abschnitt); nutzt `LineSensor`, `DriveController` und `WheelSpeedSensor` per Referenz

`LineFollower` folgt demselben Muster wie `DriveController`: Konstruktor mit Referenzen auf die anderen Objekte, `begin()`, und `update()` in jedem `loop()`.

Programmieransatz

Der Roboter berechnet aus den drei Sensoren eine Linienposition, lenkt proportional dagegen und wechselt über einen kleinen Zustandsautomaten zwischen Folgen, Lücke überbrücken und Suchen.

1. Position der Linie bestimmen

`LineSensor::getPosition()` mittelt die Sensoren, die Schwarz sehen (links = −1, Mitte = 0, rechts = +1). Bei einer 45 mm breiten Linie sehen oft zwei Sensoren gleichzeitig Schwarz, dadurch gibt es Zwischenstufen.

Sensoren L M R	Position	Bedeutung
`. # .`	0	Linie mittig, geradeaus
`# # .`	−0,5	Linie leicht links
`# . .`	−1	Linie weit links
`. # #`	+0,5	Linie leicht rechts
`. . #`	+1	Linie weit rechts

Sensoren L M R	Position	Bedeutung
`# # #`	0	Breite Stelle oder Kreuzung: geradeaus
`. . .`	–	Keine Linie: Lücke oder verloren

2. Lenken (P-Regler, optional D-Anteil)

Aus der Position wird ein Lenkwert berechnet, der die Grundgeschwindigkeit auf beide Räder verteilt. Liegt die Linie rechts (Position positiv), dreht das linke Rad schneller und der Roboter lenkt nach rechts.

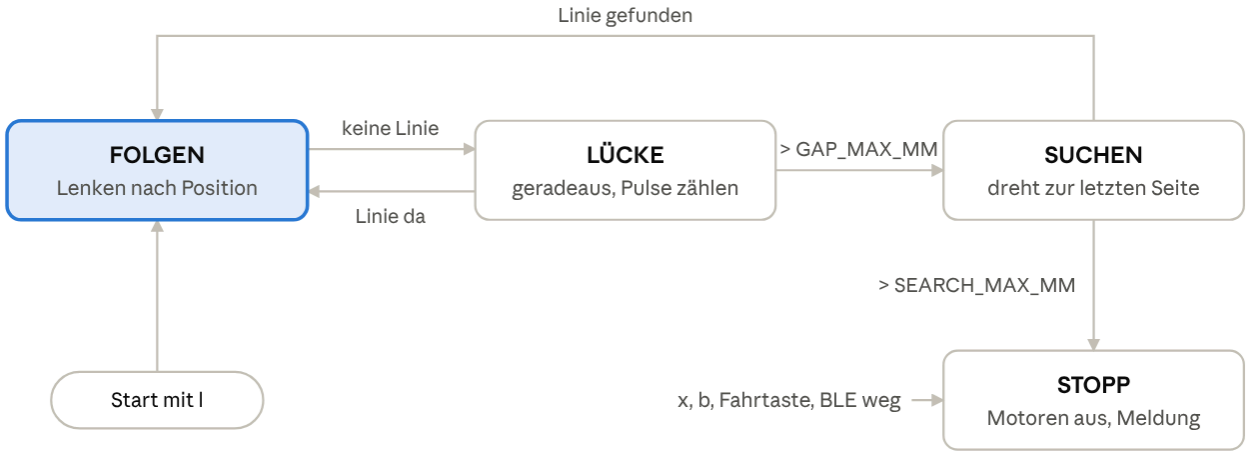
```
\begin{aligned}
\text{lenk} &= K_p \cdot \text{pos} + K_d \cdot \frac{\text{pos} -
\text{pos}_{alt}}{\Delta t} \\
\text{links} &= \text{basis} \cdot (1 + \text{lenk}) \\
\text{rechts} &= \text{basis} \cdot (1 - \text{lenk})
\end{aligned}
```

Beide Werte werden auf −100 … +100 % begrenzt und über `DriveController::drive()` gesetzt. Der D-Anteil dämpft Pendeln und bleibt anfangs 0.

3. Zustände

- **FOLGEN:** Normalbetrieb mit dem Lenkregler oben. Merkt sich die letzte Position (`pos_alt`) für die Suche.
- **LÜCKE:** Alle Sensoren sehen Weiß. Der Roboter fährt mit dem letzten geraden Kurs weiter und zählt dabei die Pulse der Lichtschranken. Taucht die Linie innerhalb von `GAP_MAX_MM` wieder auf, geht es zurück zu FOLGEN. So wird die Kreuzung überquert.
- **SUCHEN:** Lücke war zu lang. Der Roboter dreht sich auf der Stelle in die Richtung, in der die Linie zuletzt war (Vorzeichen von `pos_alt`).
- **STOPP:** Nichts gefunden nach `SEARCH_MAX_MM` Drehweg, oder Linienfolge per BLE beendet. Motoren aus, Meldung per BLE.

Ohne Linie fährt er erst geradeaus, dann sucht er



Zustände der Linienfolge · 4 Zustände

Verschwindet die Linie, fährt er zuerst geradeaus weiter; erst wenn die Lücke länger als `GAP_MAX_MM` ist, beginnt die Suche.

Kreuzung: Direkt nach einer Lücke kann der schräge Ast des X zuerst unter einem äußeren Sensor auftauchen. Damit der Roboter nicht in den falschen Ast abbiegt, wird für die ersten `CROSS_HOLD_MM` nach der Lücke die Lenkung stark begrenzt (nur sanfte Korrektur), bis die Linie wieder mittig liegt.

Ablauf in loop() und Zusammenspiel

Die Linienfolge ist ein zusätzlicher Betriebsmodus: Ist sie aus, fährt der Roboter wie bisher per Taste; ist sie an, übernimmt `LineFollower` die Fahrbefehle. Die Reihenfolge in `loop()` bleibt fast gleich:

1. `handleCommands()` – Befehle von USB und BLE lesen (u. a. Linienfolge an/aus). Eine Fahrtaste beendet die Linienfolge sofort (manuelle Übernahme).
2. `sonar.update()` – Abstand messen.
3. `lineFollower.update()` – nur im Linienmodus: Sensoren lesen, Zustand weiterschalten, `drive.drive(links, rechts)` setzen.
4. `handleObstacle()` – unverändert: drosselt per `setSpeedScale()` und bremst bei ~5 cm. Steht der Roboter wegen eines Hindernisses, pausiert `LineFollower` und fährt weiter, sobald der Weg frei ist.
5. `drive.update()` – Gleichlaufregelung und PWM.
6. Alle 200 ms die Statuszeile an USB und BLE, ergänzt um Modus und Zustand (z. B. `Modus: Linie/FOLGEN`).

Gleichlaufregelung: `DriveController` hält das befohlene Verhältnis links:rechts ein und gleicht so Motorunterschiede auch in Kurven aus. Weil die Position nur fünf Stufen hat, ändert sich der Sollwert selten und die Regelung kann arbeiten. Falls der Roboter dadurch träge lenkt oder pendelt, wird sie im Linienmodus mit `setSyncEnabled(false)` abgeschaltet.

Halte-Steuerung: Die Regel „nur fahren, solange die Taste gehalten wird“ gilt nur für manuelles Fahren. Die Linienfolge läuft ohne gehaltene Taste und wird mit `x`, `b`, einer Fahrtaste oder beim Abreißen der BLE-Verbindung beendet.

Parameter und Bedienung per BLE

Alle Werte sind Startwerte zum Einstellen; die meisten lassen sich live per BLE ändern und werden danach fest im Code eingetragen.

Parameter	Startwert	Wirkung	Live per BLE
`BASE_SPEED`	40 %	Grundtempo auf der Linie	`+` / `-` (±10 %)
`LINE_KP`	0,6	Stärke der Lenkung	`k` / `K` (±0,1)
`LINE_KD`	0	Dämpfung gegen Pendeln	`m` / `M` (±0,05)
`GAP_MAX_MM`	100 mm	So weit wird ohne Linie geradeaus gefahren (Kreuzung)	–
`CROSS_HOLD_MM`	50 mm	Strecke mit begrenzter Lenkung nach einer Lücke	–
`SEARCH_SPEED`	40 %	Drehtempo beim Suchen	–
`SEARCH_MAX_MM`	400 mm	Maximaler Drehweg (Radstrecke) beim Suchen, dann STOPP	–

Neue Befehle (USB und BLE, zusätzlich zu den bisherigen):

Taste	Funktion
`l`	Linienfolge starten / beenden
`k` / `K`	Lenkstärke Kp −/+
`m` / `M`	Dämpfung Kd −/+

Taste	Funktion
`g`	Alle Einstellungen anzeigen (jetzt inkl. Linienwerte)
`w` `a` `s` `d` `x` `b`	Beenden die Linienfolge und übernehmen manuell

Die Statuszeile zeigt zusätzlich Modus und Zustand, z. B. `Modus: Linie/LÜCKE 37 mm`, damit man beim Testen sieht, was der Roboter gerade denkt.

Testplan und offene Fragen

Wir testen in kleinen Schritten, jeweils per BLE mit Blick auf die Statuszeile.

- [ ] **Sensor prüfen:** Roboter von Hand quer über die Linie schieben, `#` muss von links über Mitte nach rechts wandern (`Linie: #.. → ##. → .#. → .## → ..#`).
- [ ] **Gerade Linie:** Linienfolge mit `l` starten, 30 % Tempo. Pendelt er → `k` ; reagiert er zu träge → `K` .
- [ ] **Kurven:** Ganze Schleife ohne Kreuzung. Fliegt er aus engen Kurven → Tempo runter oder Kp hoch.
- [ ] **Kreuzung:** Langsam über das X fahren und in der Statuszeile die Lückenlänge ablesen. `GAP_MAX_MM` auf etwa das 1,5-Fache setzen.
- [ ] **Linie verloren:** Roboter neben die Linie stellen, Suche muss in die richtige Richtung drehen und nach `SEARCH_MAX_MM` stoppen.
- [ ] **Hindernis:** Hand auf die Linie halten, er muss bremsen und nach dem Wegnehmen weiterfahren.
- [ ] **Tempo steigern:** Schrittweise mit `+` , bis er die Runde gerade noch sicher schafft; dann mit `M` (Kd) gegen Pendeln dämpfen.
- [ ] **Werte festschreiben:** Gefundene Werte (`g`) fest in `main.cpp` eintragen.

Offene Fragen (vor dem Programmieren klären):

- Wie weit liegen die drei Sensoren auseinander (mm)? Davon hängt ab, ob bei 45 mm Linie zwei Sensoren gleichzeitig Schwarz sehen.
- Wie weit sitzt der Sensor vor der Radachse? Je weiter vorne, desto früher kann er lenken.
- Wie lang ist die Lücke in der Mitte der Kreuzung (mm)?
- Soll der Roboter an der Kreuzung immer geradeaus fahren? (Annahme in diesem Konzept: ja.)
- Soll er nach dem Start eine bestimmte Anzahl Runden fahren oder endlos?